Kubernetes Cluster Setup

Prepared By: *Aditya Pratap Singh*

Project / Environment: *S3waas*

Date: *24th June 2026*

TABLE OF CONTENT

Inventory

S. No.	VM IP	Floating IP	Hostname	Configuration	OS	Role
1	172.18.31.147	10.193.117.40	node-02	16vCPU*64GB RAM	Ubuntu 22	Control-plane
2	172.18.31.167	10.193.117.45	node-1632-03	16vCPU*64GB RAM	Ubuntu 22	Control-plane
3	172.18.31.17	10.193.117.32	node-1632-04	16vCPU*64GB RAM	Ubuntu 22	Control-plane
4	172.18.31.245	10.193.117.17	node-3264-32	32vCPU*64GB RAM	Ubuntu 22	Worker
5	172.18.31.120	10.193.117.55	node-3264-31	32vCPU*64GB RAM	Ubuntu 22	Worker
6	172.18.31.72	10.193.117.18	node-3264-33	32vCPU*64GB RAM	Ubuntu 22	Worker
7	172.18.31.28	10.193.117.39	node-3264-34	32vCPU*64GB RAM	Ubuntu 22	Worker
8	172.18.31.183	10.193.117.57	node-3264-14	32vCPU*64GB RAM	Ubuntu 22	Worker
9	172.18.31.93	10.193.117.23	node-3264-11	32vCPU*64GB RAM	Ubuntu 22	Worker

Installing the Container Engines

Before running applications, every server needs the underlying "engine" that allows it to spin up software containers. We use containerd as the engine and nerdctl as a tool to control it.

1.1 Put nerdctl on the Master Nodes

```
# 1. Download the tool to your local computer
wget
https://github.com/containerd/nerdctl/releases/download/v2.2.1/nerdctl-2.2.
1-linux-amd64.tar.gz

# 2. Copy the tool to each master node
rsync -Pv /root/nerdctl root@172.18.31.147:/usr/bin/
rsync -Pv /root/nerdctl root@172.18.31.167:/usr/bin/
rsync -Pv /root/nerdctl root@172.18.31.17:/usr/bin/

# 3. Give the tool permission to run on each master node
ssh root@172.18.31.147 'chmod 755 /usr/bin/nerdctl'
ssh root@172.18.31.167 'chmod 755 /usr/bin/nerdctl'
ssh root@172.18.31.17 'chmod 755 /usr/bin/nerdctl'
```

1.2 Install the containerd Engine Everywhere

```
# Run this on EVERY master and worker node:
apt-get -y install aptitude
aptitude -y install containerd
systemctl enable containerd
systemctl start containerd
```

Building the Secure Vault (etcd)

The master nodes need a shared database to track cluster data. We run this inside an isolated, secure bubble.

2.1 Create Secure Digital Passports (Certificates)

```
# Generate the secret lock and key files
openssl genpkey -out etcd-server.key -algorithm rsa -pkeyopt
rsa_keygen_bits:4096 -pkeyopt rsa_keygen_pubexp:99999

openssl rsa -in etcd-server.key -pubout > etcd-server.pem

openssl req -new -key etcd-server.key -out etcd-server.csr -config
etcd-server.cfg -extensions my

openssl x509 -req -days 3650 -in etcd-server.csr -CA ca.crt -CAkey ca.key
-CAcreateserial -out etcd-server.crt -extensions my -extfile
etcd-server.cfg
```

2.2 Upload the Database Software & Set Permissions

Send the database installer package to the masters, unpack it, and lock down the file security permissions so unauthorized users can't read the keys.

```
# Send the files to the masters
rsync -Pv /home/aditya/Downloads/etcd-20251208-0.0.txz
root@172.18.31.147:/tmp
rsync -Pv /home/aditya/Downloads/etcd-20251208-0.0.txz
root@172.18.31.167:/tmp
rsync -Pv /home/aditya/Downloads/etcd-20251208-0.0.txz
root@172.18.31.17:/tmp

# Load the database software into the container engine on each master
ssh root@172.18.31.147 'xz -dc /tmp/etcd-20251208-0.0.txz | nerdctl load'
ssh root@172.18.31.167 'xz -dc /tmp/etcd-20251208-0.0.txz | nerdctl load'
ssh root@172.18.31.17 'xz -dc /tmp/etcd-20251208-0.0.txz | nerdctl load'
```

Now, log into **each master node** and tighten the security permissions on the keys exactly like this:

```
chmod 755 /etc/s3waas-cluster-certs/
chmod 644 /etc/s3waas-cluster-certs/ca.crt
/etc/s3waas-cluster-certs/etcd-server.crt
/etc/s3waas-cluster-certs/etcd-client.crt
chown root:root /etc/s3waas-cluster-certs/ca.crt
/etc/s3waas-cluster-certs/etcd-server.crt
/etc/s3waas-cluster-certs/etcd-client.crt
chmod 660 /etc/s3waas-cluster-certs/etcd-server.key
chown 100:0 /etc/s3waas-cluster-certs/etcd-server.key
chmod 664 /etc/s3waas-cluster-certs/etcd-client.key
chown root:101 /etc/s3waas-cluster-certs/etcd-client.key
```

2.3 Start the Database Cluster

To get the nodes talking for the first time, log into each master node, open a command-line "shell" inside the database container, change to the safe database user (`etcd`), and start them up.

```
# Run this on all three masters to step inside the database container
environment:
nerdctl run --network host -ti --name etcd-shell --rm --entrypoint
/bin/bash -v /etc/s3waas-cluster-certs/:/etc/s3waas-cluster-certs/ -v
/srv/etcd:/srv/etcd trixie/etcd:20251208-0.0
su -s /bin/bash etcd
```

While inside the container shell, run the startup command matching that specific node:

On Master 1 (`172.18.31.147`):

```
etcd --name node-02 --initial-cluster-state new --initial-cluster
node-02=http://192.168.122.10:60582,node-1632-03=http://192.168.122.11:6058
2,node-1632-04=http://192.168.122.3:60582 --initial-advertise-peer-urls
http://192.168.122.10:60582 --initial-cluster-token k0s-district
--listen-client-urls https://192.168.122.10:60581,https://127.0.0.1:60581
--advertise-client-urls
```

```
https://192.168.122.10:60581,https://192.168.122.11:60581,https://192.168.122.3:60581 --data-dir /srv/etcd --quota-backend-bytes 8589934592 --client-cert-auth --trusted-ca-file /etc/s3waas-cluster-certs/ca.crt --cert-file /etc/s3waas-cluster-certs/etcd-server.crt --key-file /etc/s3waas-cluster-certs/etcd-server.key --listen-peer-urls http://192.168.122.10:60582
```

On Master 2 (172.18.31.167):

```
etcd --name node-1632-03 --initial-cluster-state new --initial-cluster node-02=http://192.168.122.10:60582,node-1632-03=http://192.168.122.11:60582,node-1632-04=http://192.168.122.3:60582 --initial-advertise-peer-urls http://192.168.122.11:60582 --initial-cluster-token k0s-district --listen-client-urls https://192.168.122.11:60581,https://127.0.0.1:60581 --advertise-client-urls https://192.168.122.10:60581,https://192.168.122.11:60581,https://192.168.122.3:60581 --data-dir /srv/etcd --quota-backend-bytes 8589934592 --client-cert-auth --trusted-ca-file /etc/s3waas-cluster-certs/ca.crt --cert-file /etc/s3waas-cluster-certs/etcd-server.crt --key-file /etc/s3waas-cluster-certs/etcd-server.key --listen-peer-urls http://192.168.122.11:60582
```

On Master 3 (172.18.31.17):

```
etcd --name node-1632-04 --initial-cluster-state new --initial-cluster node-02=http://192.168.122.10:60582,node-1632-03=http://192.168.122.11:60582,node-1632-04=http://192.168.122.3:60582 --initial-advertise-peer-urls http://192.168.122.3:60582 --initial-cluster-token k0s-district --listen-client-urls https://192.168.122.3:60581,https://127.0.0.1:60581 --advertise-client-urls https://192.168.122.10:60581,https://192.168.122.11:60581,https://192.168.122.3:60581 --data-dir /srv/etcd --quota-backend-bytes 8589934592
```

```
--client-cert-auth --trusted-ca-file /etc/s3waas-cluster-certs/ca.crt
--cert-file /etc/s3waas-cluster-certs/etcd-server.crt --key-file
/etc/s3waas-cluster-certs/etcd-server.key --listen-peer-urls
http://192.168.122.3:60582
```

2.4 Set Databases to Automatically Run Non-Stop

Once you verify that the nodes are successfully communicating, close those manual command shells. Run these permanent background commands on the respective master nodes so the database runs permanently and restarts automatically if a server loses power

On Master 1 (172.18.31.147):

```
nerdctl run --restart unless-stopped --network host -d --name node-02 -v
/etc/s3waas-cluster-certs/:/etc/s3waas-cluster-certs/ -v
/srv/etcd:/srv/etcd trixie/etcd:20251208-0.0 --name node-02
--listen-client-urls https://192.168.122.10:60581,https://127.0.0.1:60581
--data-dir /srv/etcd --advertise-client-urls
https://192.168.122.10:60581,https://192.168.122.11:60581,https://192.168.1
22.3:60581 --quota-backend-bytes 8589934592 --initial-advertise-peer-urls
http://192.168.122.10:60582 --client-cert-auth --trusted-ca-file
/etc/s3waas-cluster-certs/ca.crt --cert-file
/etc/s3waas-cluster-certs/etcd-server.crt --key-file
/etc/s3waas-cluster-certs/etcd-server.key --listen-peer-urls
http://192.168.122.10:60582
```

On Master 2 (172.18.31.167):

```
nerdctl run --restart unless-stopped --network host -d --name node-1632-03
-v /etc/s3waas-cluster-certs/:/etc/s3waas-cluster-certs/ -v
/srv/etcd:/srv/etcd trixie/etcd:20251208-0.0 --name node-1632-03
--listen-client-urls https://192.168.122.11:60581,https://127.0.0.1:60581
--data-dir /srv/etcd --advertise-client-urls
https://192.168.122.10:60581,https://192.168.122.11:60581,https://192.168.1
22.3:60581 --quota-backend-bytes 8589934592 --initial-advertise-peer-urls
http://192.168.122.11:60582 --client-cert-auth --trusted-ca-file
/etc/s3waas-cluster-certs/ca.crt --cert-file
```

```
/etc/s3waas-cluster-certs/etcd-server.crt --key-file
/etc/s3waas-cluster-certs/etcd-server.key --listen-peer-urls
http://192.168.122.11:60582
```

On Master 3 (172.18.31.17):

```
nerdctl run --restart unless-stopped --network host -d --name node-1632-04
-v /etc/s3waas-cluster-certs/:/etc/s3waas-cluster-certs/ -v
/srv/etcd:/srv/etcd trixie/etcd:20251208-0.0 --name node-1632-04
--listen-client-urls https://192.168.122.3:60581,https://127.0.0.1:60581
--data-dir /srv/etcd --advertise-client-urls
https://192.168.122.10:60581,https://192.168.122.11:60581,https://192.168.1
22.3:60581 --quota-backend-bytes 8589934592 --initial-advertise-peer-urls
http://192.168.122.3:60582 --client-cert-auth --trusted-ca-file
/etc/s3waas-cluster-certs/ca.crt --cert-file
/etc/s3waas-cluster-certs/etcd-server.crt --key-file
/etc/s3waas-cluster-certs/etcd-server.key --listen-peer-urls
http://192.168.122.3:60582
```

Pre-loading the Servers (Air-Gap Setup)

Because our cluster lives in an isolated environment without a direct internet connection (an "air-gap"), we must manually send all the Kubernetes installation files and software components beforehand.

Run this loop from your deployment workstation to prep directories, create shortcut links, and distribute the core **k0s** Kubernetes engine packages across every node:

```
  # Create application directories and configure global shortcuts
  ssh root@master/worker 'mkdir -p /var/lib/k0s/images /var/lib/kubelet
/opt/k0s/bin/; ln -sf /opt/k0s/bin/k0s-v1.34.3+k0s.0-amd64 /usr/bin/k0s'

  # Copy the massive offline image bundle to the node
  rsync -zvP k0s-airgap-bundle-v1.34.3+k0s.0-amd64
root@master/worker:/var/lib/k0s/images/k0s-airgap-bundle-v1.34.3+k0s.0-amd6
4

  # Inject the master security keys
  rsync -v
/home/aditya/Documents/nks_k8scluster_setup-3rd-cluster/certs/k0s_district_
ngc/{ca,sa}.{key,crt,pub} root@master/worker:/var/lib/k0s/pki/

  # Copy the main executable file
  rsync -vP --ignore-existing --chmod 755 --chown root:root
k0s-v1.34.3+k0s.0-amd64 root@master/worker:/opt/k0s/bin/
done
```

3.1 Generate Map Blueprints (Configs)

Run your configuration playbook on your workstation and map the instructions to each distinct master node:

```
ansible-playbook /files/k0s_config_interpolate.yaml

scp
/home/aditya/Documents/nks_k8scluster_setup-3rd-cluster/configs/k0s_district/k0s_192.168.122.10.yaml  root@172.18.31.147:/etc/k0s.yaml
scp
/home/aditya/Documents/nks_k8scluster_setup-3rd-cluster/configs/k0s_district/k0s_192.168.122.11.yaml root@172.18.31.167:/etc/k0s.yaml
scp
/home/aditya/Documents/nks_k8scluster_setup-3rd-cluster/configs/k0s_district/k0s_192.168.122.3.yaml root@172.18.31.17:/etc/k0s.yaml
```

Starting the Kubernetes System

4.1 Initialize Master 1 (172.18.31.147)

Log into **Master 1** and tell Kubernetes to build the primary management control tower using our offline files:

```
k0s install controller --cri-socket remote:/run/containerd/containerd.sock
--disable-components autopilot,helm,konnectivity-server,windows-node -c
/etc/k0s.yaml --data-dir /var/lib/k0s --iptables-mode nft
--kubelet-root-dir /var/lib/kubelet --enable-worker

k0s start
```

4.2 Pre-Load Essential Background System Images

Run this task from your system workspace to inject essential internal networking and health check container components onto all servers:

```
# Push the foundational "pause" image tool into containerd's system memory
  docker save registry.k8s.io/pause:3.10.1 | ssh root@master/worker ctr
--namespace k8s.io image import -

# Tell containerd to unpack the giant airgap component archive we moved
earlier
  ssh root@master/worker 'ctr --namespace k8s.io image import
/var/lib/k0s/images/k0s-airgap-bundle-v1.34.3+k0s.0-amd64'
```

4.3 Add Master 2 and Master 3 to the Group

To ensure the system remains functional even if one master goes offline, link the remaining two office nodes to the first one.

On **Master 1**, generate an entry key pass:

```
k0s token create --role=controller --expiry 1h --data-dir /var/lib/k0s > /tmp/join-master
```

Copy that `/tmp/join-master` file over to **Master 2** and **Master 3**, and run this command on both of them to bring them online:

```
k0s install controller --cri-socket remote:/run/containerd/containerd.sock --disable-components autopilot,helm,konnectivity-server,windows-node -c /etc/k0s.yaml --data-dir /var/lib/k0s --iptables-mode nft --kubelet-root-dir /var/lib/kubelet --enable-worker --token-file /tmp/join-master

k0s start
```

Connecting the Worker Warehouses

Now that the control offices are linked and running smoothly, let's connect the worker nodes where our applications will run.

5.1 Copy Settings to Workers

```
  rsync -v --chmod 644 --chown root:root
/home/aditya/Documents/nks_k8scluster_setup-3rd-cluster/configs/k0s_district/k0s_192.168.122.10.yaml root@worker:/etc/k0s.yaml
```

5.2 Generate a Worker Join Key

On any active master, create a join key pass specifically for workers:

```
k0s token create --role=worker --expiry 1h --data-dir /var/lib/k0s >
/tmp/join-worker
```

Copy that `/tmp/join-worker` file over to your worker nodes, and run this setup command on **every single worker machine** to link it to the network:

```
k0s install worker --cri-socket remote:/run/containerd/containerd.sock
--data-dir /var/lib/k0s --iptables-mode nft --kubelet-root-dir
/var/lib/kubelet --token-file /tmp/join-worker

k0s start
```

5.3 Prioritize Kubernetes Tasks on Workers

To ensure the servers don't freeze or crash under heavy traffic, we adjust settings to prioritize core Kubernetes tasks over minor background processes:

```
  ssh root@worker 'mkdir /etc/systemd/system/k0sworker.service.d/'
  rsync -rv --chmod 644 --chown root:root
/files/configs/k0s_district/etc/systemd/system/k0sworker.service.d
root@worker:/etc/systemd/system
  ssh root@worker 'systemctl daemon-reload && systemctl restart
k0sworker.service'
```

Granting Admin Access & Storage Connections

6.1 Set Up the Management Command-Line Tool (kubectl)

Log into **Master 1** to create an account configuration profile for user admin2 so they can manage the cluster:

```
mkdir -p /home/admin2/.kube/
cp /var/lib/k0s/pki/admin.conf /home/admin2/.kube/config
chown -R admin2:admin2 /home/admin2/.kube/

# Create a typing shortcut so typing 'kubectl' automatically manages things
echo "alias kubectl='k0s kubectl --kubeconfig=/home/admin2/.kube/config'"
>> /home/admin2/.bashrc
```

To test if everything is working, log in as admin2 and look at your working network map:

```
k0s kubectl --kubeconfig=/var/lib/k0s/pki/admin.conf get no -owide
```

6.2 Set Up Helm (The App Installer Tool)

Install Helm onto **Master 1** to easily deploy pre-packaged apps:

```
rsync -Pv /usr/bin/helm root@172.18.31.147:/usr/bin/
ssh root@172.18.31.147 'chown root:root /usr/bin/helm; chmod 755
/usr/bin/helm'
```

6.3 Link Cloud Disk Storage (Cinder CSI)

Finally, run this sequence to connect your cluster to your OpenStack cloud hard drives, allowing applications to securely save persistent data:

```
for i in openstack-cacert cinder-csi-controllerplugin-rbac
cinder-csi-nodeplugin-rbac cinder-csi-cloud-conf csi-cinder-driver
cinder-csi-controllerplugin cinder-csi-nodeplugin cinder-csi-storageclass;
do
  cat
/home/aditya/Documents/nks_k8scluster_setup-3rd-cluster/yamls/k0s_ngc_distr
ict/${i}.yaml | ssh root@172.18.31.147 'k0s kubectl
--kubeconfig=/home/admin2/.kube/config create -f -'
done
```